Kiểm tra khóa 3

Hai bài thuộc hai lĩnh vực trong 6 giờ.

> Sau các chương ảnh, chữ và âm thanh, ta có nhiều công cụ, nhưng một buổi làm hai bài mới vẫn có thể rối: bài đầu huấn luyện được model, bài thứ hai chưa có file nộp; hoặc cả hai có điểm trong notebook nhưng ID lại sai khi xuất file. Buổi kiểm tra này luyện mục tiêu **giải hai bài thuộc hai lĩnh vực khác nhau trong sáu giờ**: nối đọc đề, baseline, validation và kiểm output thành một quy trình đi được đến file nộp. Khi thi, thời gian, phần cứng và luật chấm đọc từ đề; lịch và các con số dưới đây là một cách chia để luyện.

MỤC TIÊU SAU PHẦN ÔN

- Đọc hai bài, ghi input, output, metric và chọn baseline khả thi cho từng bài.
- Giải thích hợp đồng tiền xử lý ảnh, chữ, âm thanh trước khi đổi model.
- Ước điểm theo luật chấm và đọc lỗi theo từng ID để chọn một cải tiến có bằng chứng.
- Chạy lại suy luận, kiểm ID, schema và ghi kết quả cuối trong ngân sách buổi luyện.

Một bản đồ để khỏi nhầm nhiệm vụ

Các kiến trúc có thể dùng chung encoder, nhưng head, loss và metric phải theo đề. Image classification cho nhãn hoặc xác suất lớp; object detection cho box, lớp, score; segmentation cho mask. Text classification cho nhãn câu; token classification cho nhãn từng token; ASR cho transcript. Multimodal có thể là phân loại, retrieval hoặc generation, nên tên “multimodal” chưa nói file output phải là gì.

Câu hỏi cần trả lời	Kiểm trước khi train	Một lỗi nên tìm sớm
Ảnh thuộc lớp nào?	Kích thước, thứ tự channel, class mapping	Train/test resize khác nhau
Vật ở đâu?	Tọa độ box, IoU, ngưỡng, NMS	Đổi chiều rộng với chiều cao hoặc đơn vị box

Câu hỏi cần trả lời	Kiểm trước khi train	Một lỗi nên tìm sớm
Câu thuộc lớp nào?	Tokenization, length, padding, mask	Caption hoặc người dùng trùng hai tập
Âm thanh thuộc lớp nào?	Sample rate, log-mel, processor, group	Crop cùng file vào hai tập
Người nói câu gì?	Task transcribe hay translate, normalization WER	Chunk ghép lặp hoặc mất từ
Ảnh và chữ có giúp nhau không?	Pair ID, riêng từng nhánh, fusion	Ghép đúng shape nhưng sai cặp

Với mỗi bài, viết hợp đồng thành hai câu: một hàng nộp gồm gì, và metric chấm nó thế nào. Chưa biết scorer cần xác suất hay nhãn thì đọc phần định dạng trước khi chạy model lớn. Baseline là một đầu ra chấm được, không phải một cell in ra loss.

HỎI NHANH

Đề detection yêu cầu box, class và score cho từng vật. Notebook đang ghi một class cho cả ảnh, với accuracy validation 0.90. Việc đầu tiên cần làm là gì?

A Đổi head và file nộp sang dạng box, class, score

B Train thêm epoch cho accuracy của ảnh cao hơn

C Ghi một box phủ cả ảnh, kèm class đã đoán

D Giữ model, chỉ đổi tên cột cho khớp sample

Âm thanh có cùng phân nhánh: một nhãn cho cả clip, một transcript, hay một dãy nhãn theo thời gian khi đề hỏi sự kiện xảy ra lúc nào.

HỎI NHANH

Đề âm thanh: mỗi clip dài 10 giây, cần nộp các khoảng (bắt đầu, kết thúc) có tiếng chó sủa, chấm theo độ trùng thời gian với khoảng thật. Baseline phân loại cả clip "có hay không có chó sủa" đạt accuracy 0.95. Còn thiếu gì?

A Không thiếu gì, clip có chó sủa thì nộp khoảng 0–10 giây

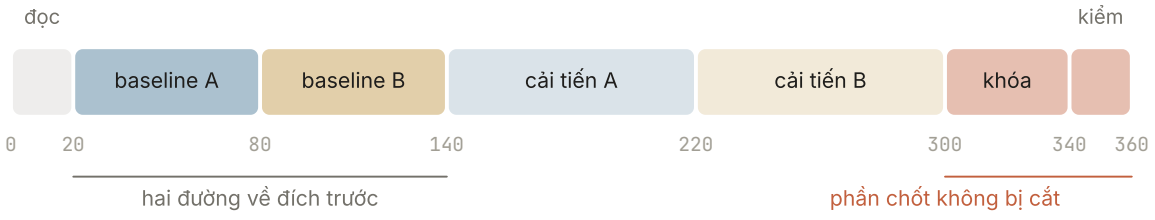
B Dự đoán theo từng frame, rồi gộp các frame thành khoảng

C Đẩy accuracy theo clip lên trên 0.99 rồi mới nộp

D Dùng nhiều mel band hơn để thấy rõ tiếng sủa

Sáu giờ mẫu: cả hai bài đều cần một đường về đích

Một lịch mẫu có 20 phút đọc hai đề, 60 phút dựng baseline A, 60 phút dựng baseline B, rồi mỗi bài 80 phút cải tiến. 40 phút tiếp theo khóa model và chạy suy luận, 20 phút cuối kiểm file. Tổng là $20 + 60 + 60 + 80 + 80 + 40 + 20 = 360$ phút. Đề cho ngân sách khác thì giữ thứ tự ưu tiên: hai baseline trước, phần chốt cuối giữ nguyên, phần cải tiến co giãn.



Lịch mẫu 360 phút: đọc hai đề, dựng hai baseline, mỗi bài một khoảng cải tiến, rồi 60 phút cuối khóa model, suy luận và kiểm file. Khi trễ, chỉ phần cải tiến co lại.

Nếu pipeline A hỏng, đừng dùng cả buổi đổi model để né việc nhìn lỗi. Thu nhỏ về vài sample: input có đúng shape không, label có đúng lớp không, scorer có đọc được output

không? Baseline A chạy rồi thì chuyển sang B trước một lượt huấn luyện lớn. Hai baseline tạo hai mốc chấm và cho biết bài nào cần thời gian hơn.

HỎI NHANH

Theo lịch mẫu, baseline A xong ở phút 80. Thực tế A xong ở phút 140, và đội vẫn giữ 60 phút cuối cho suy luận và kiểm file. Còn bao nhiêu phút cho baseline B và cải tiến, và nên làm gì trước?

A 220 phút, nên cải tiến A trước khi đụng tới B

B 220 phút, nên dựng baseline B trước

C 160 phút, nên cải tiến A trước vì A đã chạy

D 160 phút, nên dựng baseline B trước

Thứ tự này có lý do trong cách chấm. Nhiều đề quy điểm về thang chung giữa baseline của đề và bài tốt nhất; vòng 2 USA-NA-AIO 2026 dùng $S = \max\left(0, \dfrac{X - X_{\text{base}}}{X_{\text{best}} - X_{\text{base}}}\right)$, trong đó X là metric của bài nộp: kém hơn baseline thì được 0. Công thức dùng được cả cho metric càng thấp càng tốt như WER, vì khi đó tử và mẫu cùng đổi dấu. Ví dụ dưới giả sử bài A chấm macro-F1 (baseline 0.50, tốt nhất 0.90), bài B chấm WER (baseline 0.40, tốt nhất 0.15).

Python

```python
def normalized_score(x, baseline, best):
    return 100 * max(0.0, (x - baseline) / (best - baseline))

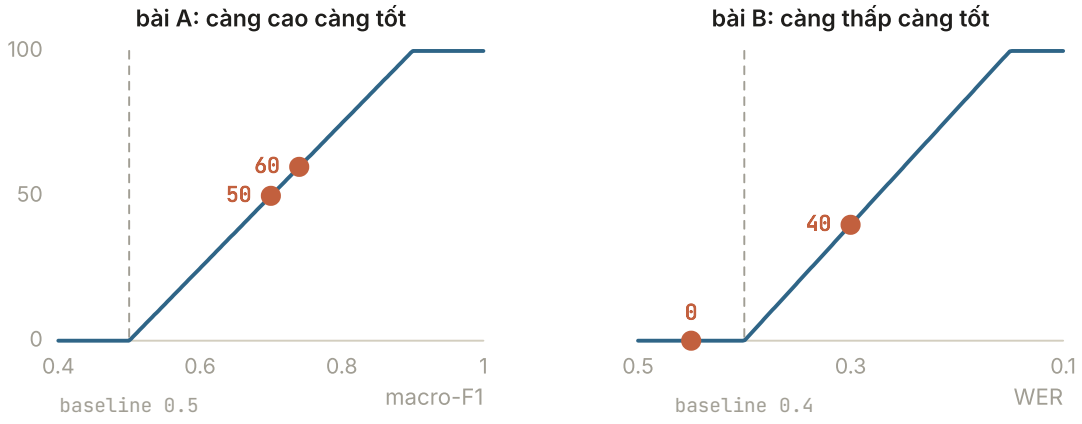
print(f"A, macro-F1 0.70 -> {normalized_score(0.70, 0.50, 0.90):.1f}")
print(f"A, macro-F1 0.74 -> {normalized_score(0.74, 0.50, 0.90):.1f}")
print(f"B, WER 0.30      -> {normalized_score(0.30, 0.40, 0.15):.1f}")
print(f"B, WER 0.45      -> {normalized_score(0.45, 0.40, 0.15):.1f}")
```

KẾT QUẢ

```
A, macro-F1 0.70 -> 50.0
A, macro-F1 0.74 -> 60.0
```

```
B, WER 0.30      -> 40.0
B, WER 0.45      -> 0.0
```

Giả sử đến phút 200, A đạt 0.70 còn B chưa có file. Tám mươi phút cải tiến A lên 0.74 thêm 10 điểm; cùng thời gian, một baseline B đạt WER 0.30 thêm 40 điểm. Một file B kém baseline của đề (WER 0.45) vẫn được 0, nên đường về đích phải vượt baseline đề cho, không chỉ hợp lệ. X_{best} chưa biết lúc thi; một ước lượng thô đủ để so hai lựa chọn.



Điểm chuẩn hóa của hai bài trong ví dụ: kém baseline của đề (vạch đứt) thì được 0. Bài A tính theo macro-F1, bài B theo WER nên trục đảo chiều; các chấm là bốn trường hợp trong code.

HỎI NHANH

Vẫn luật chấm như trong ví dụ, với bài B có baseline WER 0.40 và bài tốt nhất 0.15. Baseline B nhanh nhất ước đạt WER 0.38. Bài B được bao nhiêu điểm?

A 8

B 92

C 38

D 0

Sau khi có baseline, viết một giả thuyết: “clip dài bị mất từ ở ranh giới”, “ảnh nhỏ có đối tượng bị resize mất”, hoặc “mean pooling bỏ thứ tự”. Chọn một thay đổi kiểm được. Giữ

split, metric và tập sample khi so trước và sau; thay đồng thời model, split và normalization thì không biết cải tiến đến từ đâu.

Ví dụ kiểm output: ID đúng trước điểm tốt

Bài ảnh có ba sample; suy luận theo batch trả về dictionary từ ID sang class, với thứ tự khác manifest. Dự đoán và nhãn reference dưới đây được đặt tay để kiểm riêng khâu xuất file: khâu này phải đúng với bất kỳ model nào.

Python

```python
import numpy as np

expected_ids = ["img-a", "img-b", "img-c"]
prediction_by_id = {"img-c": 1, "img-a": 2, "img-b": 0}
assert len(expected_ids) == len(set(expected_ids))
assert set(prediction_by_id) == set(expected_ids)
ordered_predictions = np.array([prediction_by_id[sample_id] for sample_id in expected_ids])
assert ordered_predictions.shape == (3,)
assert np.isin(ordered_predictions, [0, 1, 2]).all()
reference_labels = np.array([2, 1, 1])
print(ordered_predictions)
print(f"accuracy minh hoa = {np.mean(ordered_predictions == reference_labels):.3f}")
```

KẾT QUẢ

```
[2 0 1]
accuracy minh hoa = 0.667
```

Ta đọc `[2 0 1]` theo `img-a, img-b, img-c`: đúng hai trong ba nhãn. Nếu xuất dictionary theo insertion order, các hàng đi theo `img-c, img-a, img-b` và điểm sai dù prediction theo ID vẫn đúng. Với xác suất, kiểm thêm shape `(N, C)`, số hữu hạn, thứ tự cột lớp và tổng mỗi hàng nếu đề yêu cầu; với mask hay box, kiểm theo schema riêng.



Cùng dictionary dự đoán theo ID của bài: ghi theo insertion order thì mỗi hàng nộp mang dự đoán của ID khác (đúng 0/3); sắp theo manifest thì đúng 2/3 như code.

Âm thanh thêm một bẫy: clip dài thường được cắt thành nhiều đoạn để suy luận, nên một ID sinh ra nhiều hàng.

HỎI NHANH

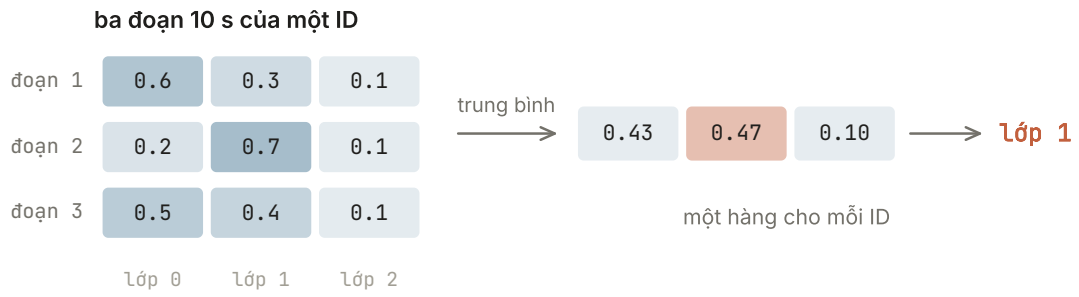
Mỗi file test dài 30 giây được cắt thành ba đoạn 10 giây cho khớp độ dài lúc train. Vì vậy bảng dự đoán có ba hàng cho mỗi ID, mỗi hàng là xác suất của ba lớp. Sample submission có một hàng mỗi ID. Ghi file nộp thế nào?

A Giữ cả ba hàng, vì scorer tự lấy trung bình theo ID

B Gọi `drop_duplicates("id")`, giữ nhãn của đoạn đầu

C Gộp xác suất theo ID, lấy argmax, rồi ghép theo sample

D Bỏ phép cắt, đưa nguyên clip 30 giây vào model



Một file 30 giây cắt thành ba đoạn 10 giây (xác suất đặt tay): trung bình xác suất ba đoạn theo ID rồi argmax, để file nộp có đúng một hàng cho ID ấy.

Ví dụ kiểm metric: WER không phải accuracy câu

Ở bài ASR, đối chiếu scorer trên vài chuỗi ngắn trước khi chạy nhiều clip. Hai reference có tổng sáu token theo quy ước tách khoảng trắng; transcript đầu thiếu một token. Hàm `token_errors` là edit distance theo token, đúng quy hoạch động của bài 7.3.

Python

```
def token_errors(reference, prediction):
    left, right = reference.lower().split(), prediction.lower().split()
    previous = list(range(len(right) + 1))
    for row, token in enumerate(left, start=1):
        current = [row]
        for column, predicted_token in enumerate(right, start=1):
            current.append(min(
                previous[column] + 1,
                current[column - 1] + 1,
                previous[column - 1] + (token != predicted_token),
            ))
        previous = current
    return previous[-1], len(left)

references = ["em học ai", "mai thi toán"]
transcripts = ["em học", "mai thi toán"]
counts = [token_errors(r, p) for r, p in zip(references, transcripts)]
total_errors = sum(error_count for error_count, _ in counts)
total_tokens = sum(length for _, length in counts)
print(counts)
```

```
print(f"corpus WER = {total_errors / total_tokens:.3f}")
print(f"cau dung hoan toan = {sum(r == p for r, p in zip(references, transcripts))}/2")
```

KẾT QUẢ

```
[(1, 3), (0, 3)]
corpus WER = 0.167
cau dung hoan toan = 1/2
```

WER bằng $1/6 \approx 0.167$, còn tỉ lệ câu đúng hoàn toàn là $1/2$. WER đếm token sai trên toàn tập; tỉ lệ câu đúng coi câu thiếu một token và câu sai hết như nhau. Hai metric có thể xếp hạng hai model ngược nhau, nên chọn model bằng đúng metric của đề. Chuẩn hóa văn bản trước khi tách token (chữ hoa, dấu câu, số) cũng là một phần của scorer: scorer local phải làm đúng như đề, không hơn, không kém.

HỎI NHANH

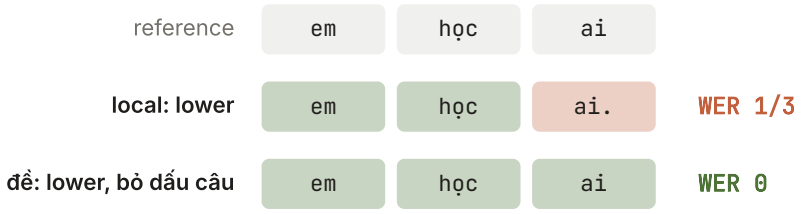
Scorer local là hàm `token_errors` ở trên. Scorer của đề thì hạ chữ thường và bỏ dấu câu trước khi tách token. Reference là `em học ai`, transcript là `Em học AI.`. WER theo scorer local và theo scorer của đề lần lượt là bao nhiêu?

A 2/3 và 0

B 1/3 và 0

C 1/3 và 1/3

D 0 và 0



transcript Em học AI. sau hai cách chuẩn hóa

Transcript “Em học AI.” qua hai scorer: scorer local chỉ hạ chữ thường nên còn token “ai.” khác “ai” (WER 1/3); scorer của đề bỏ thêm dấu câu nên khớp hoàn toàn (WER 0).

Đọc lỗi để chọn cải tiến

Với ảnh, xem một nhóm mẫu sai cùng loại: vật nhỏ, nền lạ, ảnh xoay hay box lệch. Với chữ, xem câu dài, token bị cắt, nhãn ít mẫu hoặc domain mới. Với âm, xem sample rate, clip im lặng, tiếng nền và độ dài. Với multimodal, đo từng nhánh riêng, kiểm cặp và thử thiếu nguồn. “Model không tốt” là một kết luận quá rộng để quyết định bước tiếp.

Một kết quả nên ghi ít nhất: tên thử nghiệm, hash hoặc cấu hình input, split, checkpoint và seed, metric, thời gian suy luận và nhận xét lỗi. Điểm tăng rất nhỏ trên validation ít mẫu cần được đọc theo từng mẫu, hoặc kiểm trên fold khác khi ngân sách cho phép. Không chọn model vì một seed tình cờ tốt rồi bỏ qua chi phí chạy.

Đọc theo ID cũng là cách đọc một bảng ablation. Dưới đây là dự đoán của ba cấu hình trên 25 mẫu validation của một bài ảnh + chữ ba lớp: chỉ ảnh, chỉ chữ, và fusion.

Python

```python
import numpy as np

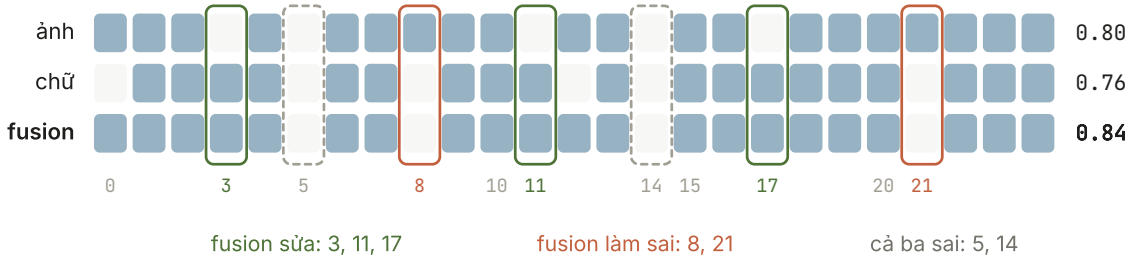
labels      = np.array([0, 1, 2, 1, 0, 2, 2, 0, 1, 1, 0, 2, 1, 0, 2, 1, 0, 2, 0, 1, 2, 0, 1,
2, 0])
image_pred  = np.array([0, 1, 2, 2, 0, 1, 2, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 2, 0, 1,
2, 0])
text_pred   = np.array([1, 1, 2, 1, 0, 1, 2, 0, 0, 1, 0, 2, 2, 0, 1, 1, 0, 2, 0, 1, 2, 1, 1,
2, 0])
fusion_pred = np.array([0, 1, 2, 1, 0, 1, 2, 0, 0, 1, 0, 2, 1, 0, 0, 1, 0, 2, 0, 1, 2, 1, 1,
2, 0])
```

```
image_ok, text_ok, fusion_ok = (pred == labels for pred in (image_pred, text_pred,
fusion_pred))
for name, ok in [("anh", image_ok), ("chu", text_ok), ("fusion", fusion_ok)]:
    print(f"{name:6s} accuracy = {ok.mean():.2f}")
for name, mask in [("fusion sua", ~image_ok & fusion_ok),
                   ("fusion lam sai", image_ok & ~fusion_ok),
                   ("ca ba cung sai", ~image_ok & ~text_ok & ~fusion_ok)]:
    print(f"{name:14s}: {np.flatnonzero(mask).tolist()}")
```

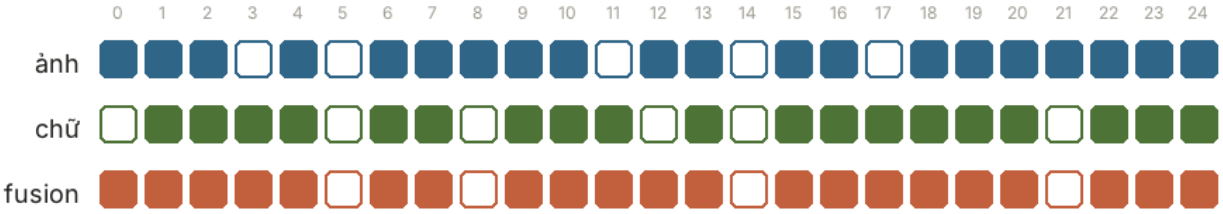
KẾT QUẢ

```
anh    accuracy = 0.80
chu    accuracy = 0.76
fusion accuracy = 0.84
fusion sua    : [3, 11, 17]
fusion lam sai: [8, 21]
ca ba cung sai: [5, 14]
```

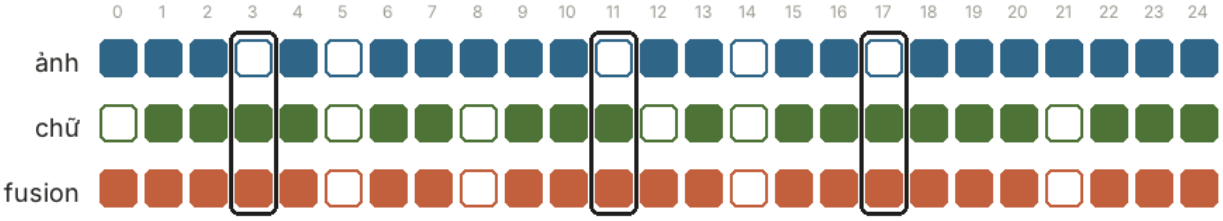
Fusion hơn ảnh 0.04, tức đúng một mẫu, nhưng năm ID đổi kết quả: sửa ba (3, 11, 17), làm sai thêm hai (8, 21). Ở ID 8 và 21 nhánh chữ sai và kéo fusion theo; đọc hai mẫu ấy cho biết nên giảm trọng số chữ hay sửa dữ liệu chữ. Hai ID 5 và 14 cả ba cấu hình cùng sai: thêm modality không cứu được, cần xem lại nhãn hoặc tìm một nguồn thông tin khác. Với 25 mẫu, một mẫu là 0.04; chênh cỡ ấy cần thêm fold hoặc thêm dữ liệu trước khi kết luận.



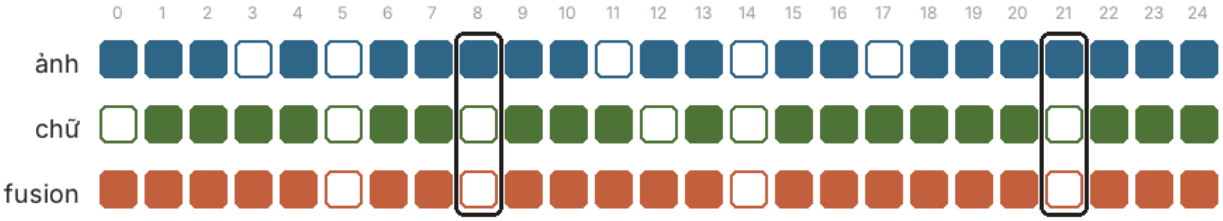
Đúng/sai theo từng ID của ba cấu hình trên 25 mẫu validation (từ code bài), ô đặc là đúng: fusion sửa ba ID nhánh ảnh sai, làm sai thêm hai ID nhánh ảnh đúng; ID 5 và 14 cả ba cùng sai.



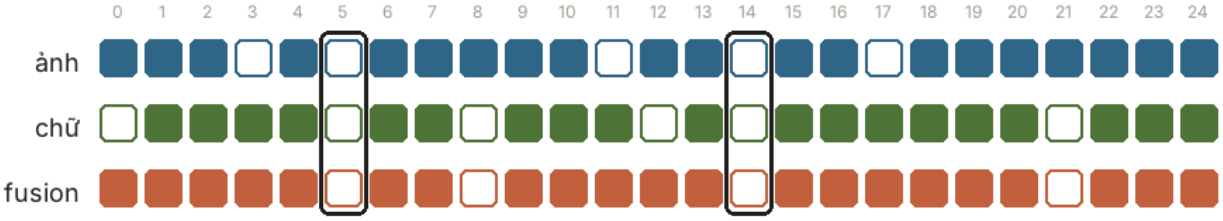
nhánh fusion: đúng 21/25 (ô đặc)



fusion sửa: ID [3, 11, 17]



fusion làm sai: ID [8, 21]



cả ba cùng sai: ID [5, 14]

Đúng/sai theo từng ID của ba cấu hình trên 25 mẫu validation của buổi kiểm tra, rồi khoanh các nhóm ID cần đọc: fusion sửa, fusion làm sai, cả ba cùng sai.

HỎI NHANH

Với 25 dự đoán ở trên, xét mọi cách gộp chỉ chọn giữa dự đoán của nhánh ảnh và của nhánh chữ ở từng mẫu. Cách gộp tốt nhất có thể, tức luôn chọn đúng nhánh đúng, đạt accuracy bao nhiêu?

A 0.92

B 0.84

C 0.96

D 1.00

Phần đọc lỗi cũng kiểm leakage. Nếu điểm quá cao, tìm sample trùng, cùng người nói, bản sao ảnh, caption chứa nhãn hoặc đoạn audio cắt từ một file chung. Điểm cao nhờ đường tắt có thể sụp khi dữ liệu test khác. Mục tiêu của buổi ôn là làm được pipeline mới, không học thuộc một bộ test.

Chốt model và kiểm ở tiến trình mới

Trong notebook, biến cũ có thể còn tồn tại dù cell nạp checkpoint chưa chạy; file nộp có thể dùng prediction của một model trước. Chạy lại từ input bằng cấu hình đã khóa: tiền xử lý → checkpoint → suy luận → xuất file → scorer hoặc bước kiểm định dạng. Kiểm ID đủ một lần, không thiếu, không trùng, không NaN, đúng cột, nhãn và schema. Một ví dụ nhỏ chạy qua toàn chuỗi trước batch lớn bắt được phần lớn lỗi ghép nối.

Checkpoint pretrained chỉ là model đã học khi weights thật được nạp cùng processor hoặc template của nó; `Model(config)` có đúng kiến trúc nhưng trọng số ngẫu nhiên. Khi asset không có trong môi trường được phép, dùng một baseline chạy được trên máy chấm; số liệu trên model card không phải điểm của pipeline. Audio có sample rate khác phải resample có lọc; mask của ảnh, chữ và âm thanh sau encoder phải theo độ dài mới. Với model sinh (VLM, LLM, ASR encoder–decoder), cấu hình sinh cũng thuộc phần phải khóa.

HỎI NHANH

Chạy lại suy luận VLM ở một tiến trình mới, cùng checkpoint, prompt và ảnh, thì 14 trong 200 câu trả lời khác với lần chạy trong notebook. Cấu hình sinh đang là `do_sample=True, temperature=0.7`. Chốt file nộp thế nào?

A Chạy thêm vài lần, nộp lần có validation cao nhất

B Giữ sampling, vì file nào cũng hợp lệ, rồi nộp lần cuối

C Sinh bằng greedy, chạy lại hai lần và so khớp từng ID

D Giảm `max_new_tokens` để câu trả lời ổn định hơn

Sau buổi luyện, kiểm xem mỗi bài đã có output chấm được, một baseline có validation, ít nhất một so sánh giải thích được và một lượt suy luận tái lập. Nếu chưa xong một bài, ghi chính xác khâu dừng: đọc dữ liệu, tiền xử lý, train, scorer hay xuất file. Buổi sau luyện đúng khâu ấy trên một bài nhỏ hơn, thay vì train lại model lớn.

Khi đi thi

- **Đọc cả hai đề trước khi code.** Với mỗi bài ghi: một hàng nộp gồm gì, metric và chiều tốt, baseline của đề, luật về asset, Internet và GPU.
- **Hai đường về đích trước, cải tiến sau.** Mỗi baseline đi hết đọc dữ liệu → model → file → scorer và vượt baseline đề cho; ước điểm chuẩn hóa để chọn bài đáng đầu tư tiếp.
- **Scorer local khớp scorer của đề.** Cùng chuẩn hóa văn bản, cùng kiểu trung bình (macro hay micro), cùng cách ghép ID; thử trên vài hàng đặt tay trước khi dùng để chọn model.
- **Đọc lỗi theo ID trước khi đổi model.** So từng mẫu giữa hai cấu hình; trên validation nhỏ, một mẫu đã là vài phần trăm.
- **Khóa sớm, chạy lại ở tiến trình mới.** Nạp checkpoint và processor đã lưu, sinh greedy, đo thời gian suy luận toàn test; không khởi động một lượt chạy dài hơn thời gian còn lại.

- **Kiểm file cuối và giữ bản dự phòng.** Đủ ID, không trùng, không NaN, giá trị thuộc tập nhãn; file đã qua kiểm thì giữ lại cho đến khi có file tốt hơn cũng đã qua kiểm.

Tóm tắt

CNN hay ViT xử lý ảnh nhưng head quyết định nhiệm vụ; tokenizer và encoder xử lý chữ nhưng mask và độ dài quyết định phần được nhìn thấy; audio encoder đọc waveform nhưng sample rate quyết định thời gian; fusion dùng nhiều nguồn nhưng ID và căn thời gian quyết định chúng nói về cùng một sự việc. Tất cả quay về split và metric để biết một thay đổi có giúp trên dữ liệu mới không.

- Hai bài cần hai baseline có đầu ra chấm được và vượt baseline của đề; dành thời gian rõ cho suy luận và kiểm file.
- Input, output và metric quyết định pipeline; scorer local phải chấm đúng như scorer của đề.
- Đọc lỗi theo từng ID và đổi một phần mỗi lượt để có bằng chứng cải tiến.
- Khóa cấu hình, kể cả cách sinh; chạy lại ở tiến trình mới và kiểm ID, schema trước khi nộp.

Kiểm tra cuối bài

CÂU 1

Đề phân loại ý định từ câu nói. Đội nối một ASR pretrained, có WER validation 0.25, với một bộ phân loại chữ. Bộ phân loại được train trên transcript do người gõ, có sẵn trong dữ liệu train. Trên transcript người gõ của validation, macro-F1 là 0.88. Cả pipeline chạy từ audio validation chỉ được 0.69. Cải tiến nào nhắm đúng chỗ hở?

A Train bộ phân loại thêm epoch trên transcript người gõ

B Train bộ phân loại trên transcript do ASR sinh ra

C Báo 0.88 là điểm của pipeline, vì ASR là phần pretrained

D Bỏ dấu khi tính WER để hai bước khớp nhau hơn

CÂU 2

Một CNN 1D trên log-mel cho mỗi frame của clip một điểm z_t, và z_t cao nghĩa là frame ấy có tiếng mưa. Theo nhãn của đề, clip dương nếu mưa chiếm ít nhất nửa thời lượng. Gộp $z_1, \ldots, z_T$ thành một điểm của clip, rồi so với ngưỡng, bằng cách nào thì khớp với định nghĩa nhãn?

A Lấy max của các z_t

B Lấy min của các z_t

C Lấy trung bình của các z_t

D Lấy trung vị của các z_t

PHỎNG THEO MIT 6.036 · Final, Spring 2022 · Q8(b)

CÂU 3

Mỗi mẫu là một video 10 giây. Nhánh ảnh lấy 25 khung hình mỗi giây, được 250 vector. Audio encoder trả 50 vector mỗi giây, được 500 vector. Để fusion theo từng bước, đội lấy 250 vector audio đầu cho khớp shape. Code chạy, hai nhánh cùng shape `(250, d)`, nhưng fusion trên validation chỉ ngang với nhánh ảnh. Nên làm gì trước?

A Kết luận audio không giúp gì và bỏ nhánh audio

B Đổi chỉ số của mỗi cặp ra giây, rồi assert hai bên trùng mốc

C Thêm assert hai nhánh cùng shape trước bước fusion

D Train fusion thêm vài epoch để nó tự học cách căn

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 266

CÂU 4

Đề hỏi đáp về ảnh có 1500 bộ (ảnh, câu hỏi, đáp án) để train, một GPU T4, và được dùng hai checkpoint pretrained: một vision encoder, và một language model chỉ đọc chữ, chưa từng được nối với nhau. Nên train phần nào?

A Fine-tune toàn bộ language model, giữ vision encoder

B Train một VLM nhỏ từ đầu cho vừa bộ nhớ T4

C Giữ nguyên hai model, chỉ học lớp nối ảnh vào language model

D Không train gì, mô tả ảnh bằng chữ rồi đưa vào prompt

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 263

CÂU 5

Một clip dài 590 giây được cắt thành 21 đoạn 30 giây, hai đoạn kề nhau chồng lên nhau 2 giây. ASR chép đúng vùng chồng ở cả hai đoạn, rồi các transcript được nối thẳng vào nhau. Mỗi vùng chồng có trung bình 5 từ, và reference có 1500 từ. So với ghép đúng, WER tăng thêm khoảng bao nhiêu?

A 0.067

B 0.133

C 0.033

D 0, vì WER không tính từ thừa

CÂU 6

Phân loại cảm xúc review sản phẩm bằng một encoder giới hạn 128 token, phần vượt bị cắt từ cuối. Đọc lỗi validation thì thấy 70% câu sai là review dài hơn 128 token, và nhiều review chốt ý ở câu cuối, như "... nhưng dùng một tuần thì hỏng". Nên thử gì trước?

A Train thêm epoch để model hiểu review dài hơn

B Bỏ các review dài khỏi tập train cho đỡ nhiễu

C Tăng `max_length` lên 1024 với cùng checkpoint

D Giữ 64 token đầu và 64 token cuối, rồi đo lại

CÂU 7

Train và validation là giọng nói thu ở 16 kHz, còn đề cho biết test là cuộc gọi điện thoại thu ở 8 kHz. Đội resample test lên 16 kHz cho khớp processor, và validation có macro-F1 0.90. Điểm này đã dự đoán được điểm test chưa, và nên làm gì?

A Được rồi, vì resample đã đưa test về đúng hợp đồng

B Được, nếu đưa thẳng audio 8 kHz vào processor

C Chưa, cần hạ validation xuống 8 kHz rồi nâng lại để đo

D Chưa, cần tăng số mel band để bù dải tần bị thiếu

CÂU 8

Đề phân loại bình luận tiếng Việt. Đội lấy một checkpoint có tokenizer tự hạ chữ thường và bỏ dấu. Bước chuẩn hóa của nó biến `Bán bàn bạn ban` thành `ban ban ban ban`. Nên làm gì?

A Đổi sang tokenizer và checkpoint giữ dấu tiếng Việt

B Giữ nguyên, vì encoder có ngữ cảnh sẽ tự phân biệt bốn từ

C Chuẩn hóa NFC trước khi đưa vào tokenizer là đủ

D Tăng `max_length` để bù các token đã bị gộp

CÂU 9

Phân loại sản phẩm từ ảnh và mô tả. Trên validation, chỉ chữ được 0.97, chỉ ảnh được 0.71, fusion được 0.97. Đọc dữ liệu thì thấy 80% mô tả trong train chứa đúng tên lớp, như "áo khoác nam ...". Đề nói mô tả ở test do người mua viết tự do. Nên làm gì trước?

A Nộp model chỉ dùng chữ, vì nó cao nhất và nhẹ nhất

B Che tên lớp trong mô tả validation, rồi đo lại từng nhánh

C Bỏ nhánh ảnh, vì fusion không hơn model chỉ dùng chữ

D Đặt trọng số ảnh 0.5 trong fusion cho cân hai nhánh

CÂU 10

Còn 20 phút. File A và file B, lấy từ checkpoint epoch 6 của B, đã qua kiểm ID và schema, và đã nằm trong thư mục nộp. Checkpoint epoch 9 của B có macro-F1 validation cao hơn 0.03, nhưng suy luận toàn bộ test đo được mất 25 phút. Nên làm gì?

A Chạy suy luận epoch 9, và nộp khi đủ hàng

B Tăng batch cho kịp, không cần đo lại bộ nhớ

C Sửa tay nhãn của vài ID khó trong B cho chắc

D Giữ file epoch 6, và ghi lại epoch 9 vào báo cáo

Đọc thêm

- scikit-learn — Model evaluation. Đối chiếu metric thay vì dùng `score` mặc định cho mọi nhiệm vụ.
- PyTorch — Saving and Loading Models. Luyện lưu, nạp lại checkpoint và đưa model về eval trước suy luận.

- IOAI 2026 — đề thi cá nhân. Mỗi task tách phần statement và phần định dạng output; luyện đọc phần định dạng trước khi chạy model.
- Các mục đọc thêm trong 7.1–7.4. Chọn nguồn theo khâu đang cần sửa: sample rate và STFT, group split, CTC và WER, hoặc fusion và template.